

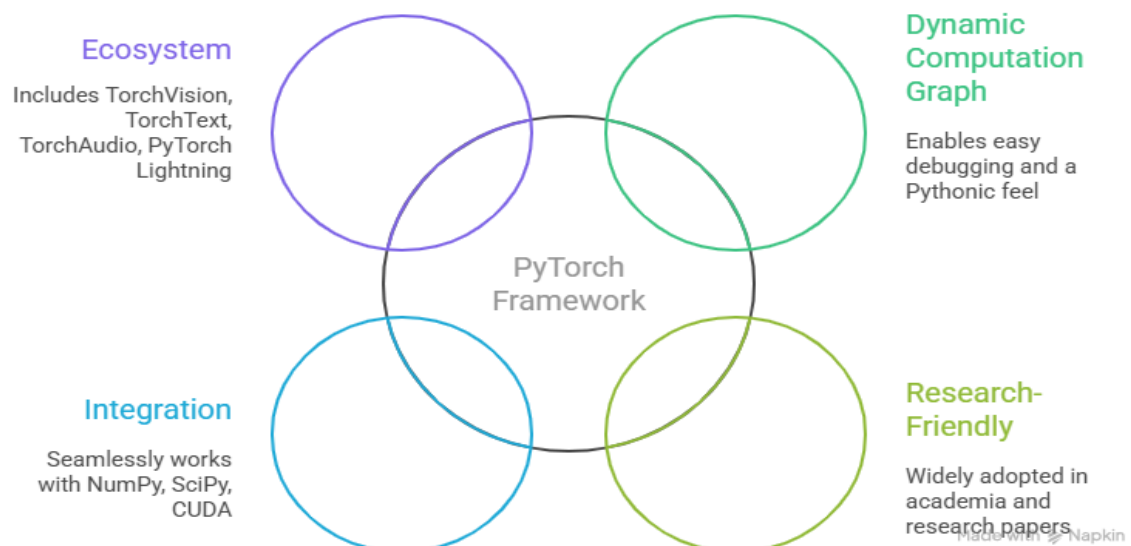
PyTorch Fundamentals: From Tensors to Neural Networks

Overview

PyTorch is an open-source deep learning framework developed by Facebook's AI Research (FAIR) lab.

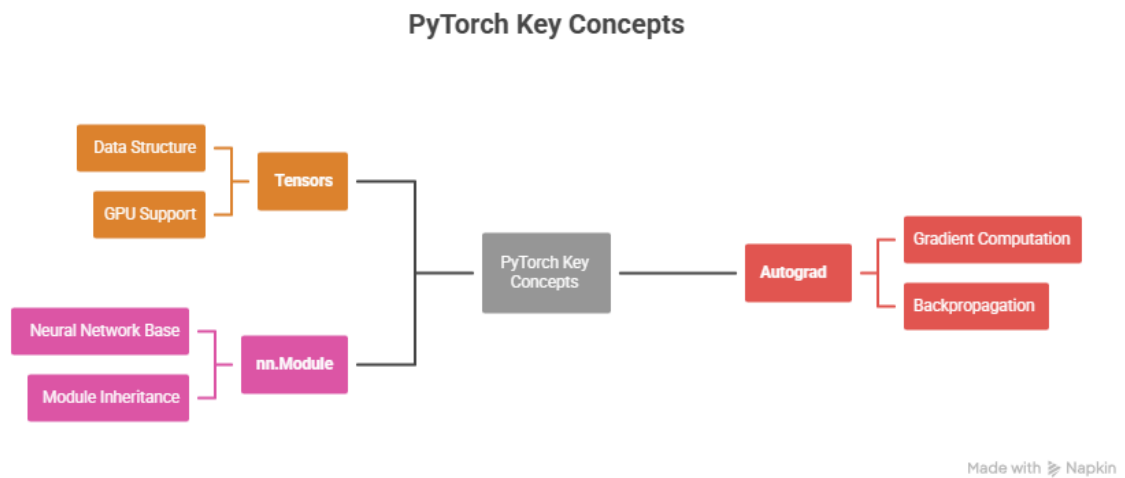
It became popular because:

- Dynamic computation graph → easy debugging, Pythonic feel.
- Research-friendly → widely adopted in academia and research papers.
- Integration → works seamlessly with NumPy, SciPy, CUDA.
- Ecosystem → TorchVision, TorchText, TorchAudio, PyTorch Lightning.



PyTorch is often called the researcher's framework, while TensorFlow is seen as the production framework.

Key Concepts



◆ Tensors

- Core data structure (like NumPy arrays but with GPU support).

- Example:

```
import torch

x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)

print(x)
```

◆ Autograd (Automatic Differentiation)

- PyTorch automatically computes gradients for backpropagation.

- Example:

```
x = torch.tensor(2.0, requires_grad=True)

y = x**2 + 3*x + 4

y.backward()

print(x.grad) # dy/dx = 7
```

◆ nn.Module

- Base class for neural networks.
- You define layers in `__init__()` and computations in `forward()`.

```
import torch.nn as nn
```

```
class SimpleNN(nn.Module):
```

```
    def __init__(self):
```

```
        super(SimpleNN, self).__init__()
```

```
        self.fc1 = nn.Linear(784, 128)
```

```
        self.fc2 = nn.Linear(128, 64)
```

```
        self.fc3 = nn.Linear(64, 10)
```

```
    def forward(self, x):
```

```
        x = torch.relu(self.fc1(x))
```

```
        x = torch.relu(self.fc2(x))
```

```
        return torch.softmax(self.fc3(x), dim=1)
```

◆ Optimizers (`torch.optim`)

- Update weights during training.

```
import torch.optim as optim
```

```
model = SimpleNN()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Building a Neural Network in PyTorch (MNIST Example)

Step 1: Load Data

```
from torchvision import datasets, transforms

from torch.utils.data import DataLoader

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,),
(0.5,))])

train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)

test_data = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)

test_loader = DataLoader(test_data, batch_size=64, shuffle=False)
```

Step 2: Define Model

```
class MNISTModel(nn.Module):

    def __init__(self):

        super(MNISTModel, self).__init__()

        self.fc1 = nn.Linear(28*28, 128)

        self.fc2 = nn.Linear(128, 64)

        self.fc3 = nn.Linear(64, 10)

    def forward(self, x):

        x = x.view(-1, 28*28) # Flatten image

        x = torch.relu(self.fc1(x))

        x = torch.relu(self.fc2(x))

        return torch.log_softmax(self.fc3(x), dim=1)
```

Step 3: Training Loop

```
criterion = nn.NLLLoss()

optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):

    for images, labels in train_loader:

        optimizer.zero_grad()

        output = model(images)

        loss = criterion(output, labels)

        loss.backward()

        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

Step 4: Evaluation

```
correct = 0

total = 0

with torch.no_grad():

    for images, labels in test_loader:

        output = model(images)

        _, predicted = torch.max(output, 1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

print("Test Accuracy:", correct / total)
```

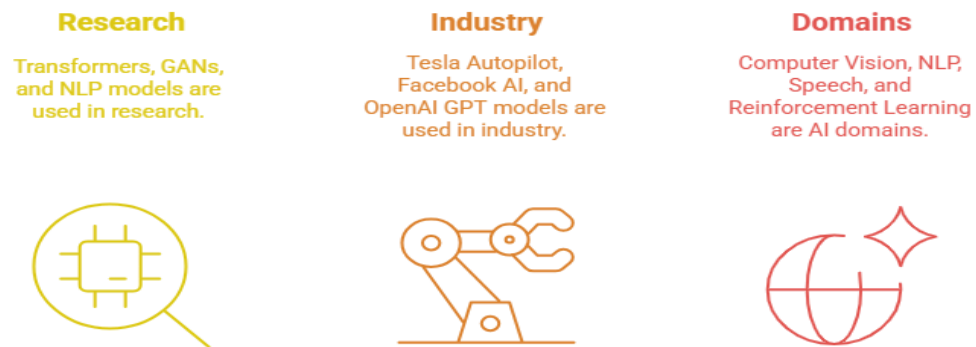
TensorFlow vs PyTorch

Aspect	TensorFlow	PyTorch
Computation	Static (TF1), Eager (TF2)	Dynamic (define-by-run)
Ease of Use	More structured	Pythonic, intuitive
Adoption	Industry & Production	Research & Academia
Deployment	TF Serving, TF Lite, TF.js	TorchServe, ONNX
Ecosystem	Mature, broader	Growing rapidly

PyTorch → Research. TensorFlow → Production.

INGRADE

Use Cases



Made with  Napkin

- Research: Transformers (BERT, GPT), GANs, NLP models.
- Industry: Tesla Autopilot (vision), Facebook AI, OpenAI GPT models.
- Domains: Computer Vision, NLP, Speech, Reinforcement Learning.

Key Takeaways

- PyTorch = dynamic, flexible, researcher-friendly.
- Core features: Tensors, Autograd, nn.Module, optim.
- Workflow = Data → Model → Training → Evaluation.
- PyTorch powers modern AI models like GPT, BERT, and GANs.